cubical-viz: Phase 3 Feature Design

Five Advanced Visualization Features

Oguri Cap

March 2026

Contents

1 Overview **2**

2 Proof Relevance Panel **2**
2.1 Motivation . . . 2
2.2 Layout . . . 2
2.3 Proof Tree Structure . . . 2
2.4 Bidirectional Hover . . . 3
2.5 Example: Transport Rule Proof Tree . . . 3
2.6 Implementation Notes . . . 3

3 Higher-Dimensional Paths: Square and Cube as Typed Terms **3**
3.1 Motivation . . . 3
3.2 Square as a Typed Term . . . 3
3.3 Cube as a Typed Term . . . 4
3.4 Implementation Notes . . . 5

4 The Face Lattice $\mathbb{F}$ **5**
4.1 Motivation . . . 5
4.2 What is the Face Lattice? . . . 5
4.3 Visualization: Hasse Diagram . . . 5
4.4 Interactive Mode . . . 6
4.5 Dimension Selector . . . 6
4.6 Implementation Notes . . . 6

5 Univalence Proof Walkthrough **6**
5.1 Motivation . . . 6
5.2 Structure of the Proof . . . 7
5.3 Multi-Page Walkthrough Design . . . 7
5.4 Scene Designs Per Stage . . . 7
5.5 Interaction . . . 7
5.6 Implementation Notes . . . 8

6 Dependency Graph Navigation **8**
6.1 Motivation . . . 8
6.2 Graph Structure . . . 8
6.3 Visual Design . . . 8
6.4 Interaction . . . 9
6.5 Access . . . 9
6.6 Implementation Notes . . . 9

7 Priority and Sequencing **10**

8 Conclusion **10**

1 Overview

This document describes the detailed visual and interaction design for five advanced features planned for `cubical-viz`, a browser-based interactive visualizer for cubical type theory. These features go beyond the individual rule illustrations completed in Phase 1 and 2 and address higher-level pedagogical goals: showing the structure of proofs, relating rules to one another, and guiding the user through complete constructions.

Features covered:

1. Proof Relevance Panel
2. Higher-Dimensional Paths (Square and Cube as Typed Terms)
3. The Face Lattice $\mathbb{F}$
4. Univalence Proof Walkthrough
5. Dependency Graph Navigation

Each section follows the same structure: motivation, typing rule(s), 3D visualization design, step breakdown, interaction model, and implementation notes.

2 Proof Relevance Panel

2.1 Motivation

In cubical type theory, paths are *proof-relevant*: two witnesses of $a = b$ are themselves paths in the space of paths, and the identity of a proof matters. The current rule pages show typing judgments and 3D scenes in isolation. The *Proof Relevance Panel* pairs each rule with a live proof tree that shows *which sub-terms were used and how they compose.*

This answers the question a student naturally asks: *where did this term come from, and what lemmas were needed?*

2.2 Layout

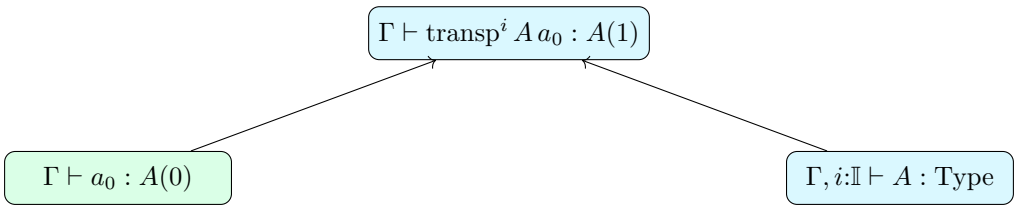
The page layout splits horizontally into two panes:

- **Left pane (60%):** the existing 3D scene with the typing rule header and step panel (unchanged).
- **Right pane (40%):** a scrollable *proof tree* rendered as an SVG tree diagram using D3.js (or a hand-rolled recursive SVG layout).

The right pane appears only when the user clicks a "Show Proof" toggle button below the typing rule. This keeps the default view uncluttered.

2.3 Proof Tree Structure

The proof tree is a top-down tree where each node is a typing judgment $\Gamma \vdash e : T$. Leaves are axioms; internal nodes are rule applications. Node layout:



Each node has two visual states:

- **Default:** dim gray border, term in KaTeX.
- **Highlighted:** colored border + glow, matching the 3D scene's hover color for the corresponding sub-term.

2.4 Bidirectional Hover

Hovering a node in the proof tree highlights the corresponding object in the 3D scene (using the existing `hoverState` store), and vice versa. This uses the same `termKey` string already defined in each rule's `termMappings`.

2.5 Example: Transport Rule Proof Tree

$$\frac{\overline{\Gamma,\, i{:}\mathbb{I} \vdash A : \mathrm{Type}} \quad \overline{\Gamma \vdash a_0 : A(i_0)}}{\Gamma \vdash \mathrm{transp}^i\, A\, [\varphi \mapsto u]\, a_0 : A(i_1)}\ (\mathrm{transp})$$

The tree collapses compound hypotheses into expandable "..." nodes to avoid overflow. Clicking a node expands its sub-derivation.

2.6 Implementation Notes

- New Svelte component: `src/lib/ProofPanel.svelte`
- Each `RuleDefinition` gains an optional `proofTree` field of type `ProofNode`:

```
interface ProofNode {
  judgment: string;     // KaTeX string
  ruleName?: string;    // label shown on the line, e.g. "transp"
  termKey?: string;     // links to 3D hover highlight
  children: ProofNode[];
}
```

- Rendered as recursive SVG: each node is a `<foreignObject>` containing KaTeX; edges are `<line>` elements.
- Horizontal rule bar drawn between children and conclusion, labeled with `ruleName`.

3 Higher-Dimensional Paths: Square and Cube as Typed Terms

3.1 Motivation

The primitives sidebar already has "Square" and "Cube" entries with placeholder scenes. This feature fills them in with conceptually accurate visualizations: a *square* is a 2-dimensional path (a path-of-paths), and a *cube* is a 3-dimensional path.

These visualizations are the geometric heart of cubical type theory: every higher homotopy is literally a higher-dimensional cube.

3.2 Square as a Typed Term

Typing rule. A square is a term $s : \mathrm{Sq}(A, p, q, r, t)$ where:

$$s : \prod_{i,j:\mathbb{I}} A \quad \text{with boundary} \begin{cases} s(i,0) = p(i) & \text{bottom face} \\ s(i,1) = q(i) & \text{top face} \\ s(0,j) = r(j) & \text{left face} \\ s(1,j) = t(j) & \text{right face} \end{cases}$$

3D visualization. The scene shows a filled square with four colored boundary paths:

- Bottom edge p: **cyan** (`INTERVAL`)
- Top edge q: **orange** (`RESULT`)
- Left edge r: **green** (`PARTIAL_DATA`)
- Right edge t: **yellow** (`BASE_POINT`)
- Interior mesh: dim translucent **pink** (`FILL_RESULT`, opacity 0.25)

The animation sweeps a horizontal scan line from $j = 0$ to $j = 1$, tracing how $s(-, j)$ evolves from path p to path q.

Steps.

1. **Boundary paths**: Display the four colored edges. Step panel explains that these are the four faces of the 2-cube.
2. **Interior sweep**: Animate the scan line. Explain that s is a homotopy from p to q relative to fixed endpoints.
3. **Full square**: Show the filled mesh. Explain that the square itself is a term of type A indexed by $\mathbb{I}^2$.

3.3 Cube as a Typed Term

Typing rule. A 3-cube is a term $c : \prod_{i,j,k:\mathbb{I}} A$ with six square faces. The faces are indexed by setting each of i, j, k to 0 or 1.

3D visualization. The scene shows a wire-frame cube (like the Kan Filling scene but fully closed) with face colors:

- Each of the 6 faces is a distinct translucent color, one per face name.
- Edges shared between faces are drawn in a neutral gray.
- A small labeled sphere floats at the interior center labeled "$c(i, j, k)$".

The animation rotates the cube slowly around the j-axis, pausing at each of the six face views to highlight that face and display its boundary constraint in the step panel.

Steps.

1. **Three dimensions**: Show the cube skeleton and explain that $i, j, k : \mathbb{I}$ each range over the interval.
2. **Six faces**: Rotate to each face in sequence; highlight the face and show its typing constraint.
3. **Interior term**: Animate a point traveling through the interior, representing $c(i, j, k)$ for varying i, j, k.

3.4 Implementation Notes

- `src/lib/primitives/Square.svelte` and `Cube.svelte` (already stubbed; fill in `setup`/`update`).
- Square interior: `PlaneGeometry` with `MeshBasicMaterial({color, transparent, opacity: 0.25})`.
- Scan line: a thin `LineSegments` object at $y = \text{lerp}(0, 2, t)$, updated each frame.
- Cube faces: 6 `PlaneGeometry` meshes, each slightly inset from the cube face to avoid z-fighting with the wireframe edges.

4 The Face Lattice $\mathbb{F}$

4.1 Motivation

The face lattice $\mathbb{F}$ is the algebraic structure that governs which faces of a cube are "active" in a partial element. A partial element $[\varphi \mapsto u]$ is defined on exactly those points satisfying the *face formula* $\varphi \in \mathbb{F}$. Understanding $\mathbb{F}$ demystifies the notation $[\varphi \mapsto u]$ that appears in every `comp`, `fill`, and `transp` rule.

4.2 What is the Face Lattice?

$\mathbb{F}$ is the free distributive lattice on generators $\{(i = 0), (i = 1) \mid i \in \text{Var}\}$ with the relation $(i = 0) \wedge (i = 1) = \bot$. Elements of $\mathbb{F}$ are disjunctions of conjunctions of face constraints.

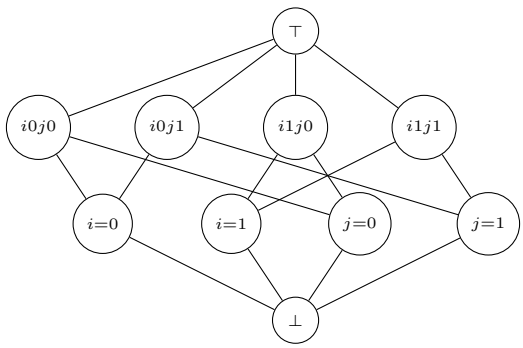
Examples (1-cube).

$$\begin{aligned} \bot &= \text{no face (empty)} \\ (i = 0) &= \text{left endpoint} \\ (i = 1) &= \text{right endpoint} \\ (i = 0) \vee (i = 1) &= \text{both endpoints} \\ \top &= \text{everywhere} \end{aligned}$$

Examples (2-cube). The 2-cube $\mathbb{I}^2$ has face lattice elements including: $(i = 0)$, $(j = 0)$, $(i = 0) \wedge (j = 0)$, $(i = 0) \vee (j = 0)$, etc.

4.3 Visualization: Hasse Diagram

The primary visualization is a **Hasse diagram** of the face lattice for the 2-cube (16 elements), drawn as an SVG or a Three.js scene with nodes and edges.



Node colors:

- $\bot$: black
- $\top$: **pink** (`FILL_RESULT`)
- Atomic faces ($i = 0$) etc.: **green** (`PARTIAL_DATA`)
- Conjunctions: **cyan** (`INTERVAL`)
- Disjunctions: **yellow** (`BASE_POINT`)

4.4 Interactive Mode

Clicking a node in the Hasse diagram highlights it and:

1. Displays the formula in the panel (e.g. "$(i = 0) \vee (j = 0)$").
2. Shows a small 2D unit square $[0, 1]^2$ with the *active region* of that formula colored in — visually demonstrating which subset of the cube this formula selects.
3. If the selected formula matches one used in another rule page (e.g. the Kan Filling rule uses $(i = 0) \vee (i = 1) \vee (j = 0)$), a "See in context" link appears to jump to that rule.

4.5 Dimension Selector

A small toggle at the top selects the dimension: **1-cube** (5 elements, trivial), **2-cube** (16 elements, default), or **3-cube** (too large to display fully; show a summarized view with collapsible subtrees).

4.6 Implementation Notes

- New page: `src/lib/FaceLattice.svelte`
- New route entry `'face-lattice'` in `types.ts` under a new sidebar category **Theory** (between Operations and Typing Rules).
- SVG-based layout: use a hand-rolled Sugiyama-style layered layout (since the lattice has a natural layer structure by rank), or adapt `d3-dag` if available via CDN.
- The 2D active-region preview uses an HTML Canvas element (100×100px) drawn next to the Hasse diagram.
- No Three.js needed for this page.

5 Univalence Proof Walkthrough

5.1 Motivation

The Univalence Axiom states that equivalent types are equal:

$$\mathrm{ua} : (A \simeq B) \to (A =_{\mathrm{Type}} B)$$

In cubical type theory, this is not an axiom but a *theorem*, proved using Glue types. The proof is a flagship construction that uses almost every feature of the system: paths, transport, Kan filling, and Glue types. A multi-step walkthrough of this proof is the most ambitious visualization in the project and serves as a capstone for students who have worked through all other pages.

5.2 Structure of the Proof

The proof has four stages:

1. **Setup**: Given $e : A \simeq B$, define a line of types $\mathrm{Glue}\,[\,i = 0 \mapsto (A, e),\ i = 1 \mapsto (B, \mathrm{id})\,]$.
2. **Glue construction**: Show that this Glue type at $i = 0$ is definitionally A and at $i = 1$ is definitionally B.
3. **Path in the universe**: The line of Glue types gives a path $i \mapsto \mathrm{GlueType}(i)$ in Type, which is exactly $A =_{\mathrm{Type}} B$.
4. **Corollary**: Transport along this path gives the underlying function of e, closing the loop.

5.3 Multi-Page Walkthrough Design

This feature is implemented as a **multi-page guided tour** rather than a single rule page. A persistent top bar displays a progress indicator:

```
[ Setup ] → [ Glue ] → [ Path in Type ] → [ Transport ]
```

Each stage is a full-page view with:

- A 3D scene (reusing and composing existing rule scenes).
- An annotation layer: arrows and labels overlaid on the 3D canvas pointing to specific objects with explanatory callouts.
- A text panel on the right with the proof fragment in KaTeX.

5.4 Scene Designs Per Stage

Stage 1: Setup. Show two spheres labeled A and B connected by a labeled arc e (like the Glue Types scene). A faint dotted line appears between them labeled $i \in [0, 1]$, previewing the path about to be constructed. Color A in **cyan**, B in **orange**.

Stage 2: Glue Construction. Morph the previous scene: the arc e thickens into a cylinder representing the Glue type. Small text sprites on the cylinder ends read "Glue(0) = A" and "Glue(1) = B". A cross-section slider lets the user drag $i \in [0, 1]$ to inspect Glue(i).

Stage 3: Path in the Universe. Zoom out to show "Type" as a large ambient space (dark background, subtle grid). The cylinder from Stage 2 appears as a curved path in this space from A to B. Label the path "$A =_{\mathrm{Type}} B$".

Stage 4: Transport. Split-screen: left shows the path in Type; right shows a point $a : A$ being transported along the path, arriving as $e(a) : B$. This reuses the TransportRule scene with appropriate labels.

5.5 Interaction

- **Stage navigation**: Prev/Next buttons in the top bar.
- **Within each stage**: the standard step panel from other rules.
- **Cross-links**: "Review: Glue Types", "Review: Transport" buttons link to the corresponding rule pages.
- **Completion**: On finishing Stage 4, a congratulations message and a summary proof in a modal window.

5.6 Implementation Notes

- New top-level route: `'univalence'` in `types.ts`.
- New component: `src/lib/Univalence.svelte` — manages the stage state and renders each stage's sub-component.
- Sub-components: `UniSetup.svelte`, `UniGlue.svelte`, `UniPath.svelte`, `UniTransport.svelte`.
- Stage 3 ambient space: large `PlaneGeometry` grid at $y = -5$ with `GridHelper`; camera pulled back to $z = 20$.
- The annotation layer is an absolutely positioned `<svg>` overlaid on the Three.js canvas, using projected 3D coordinates: `vector.project(camera)` converts world coords to clip space for overlay positioning.

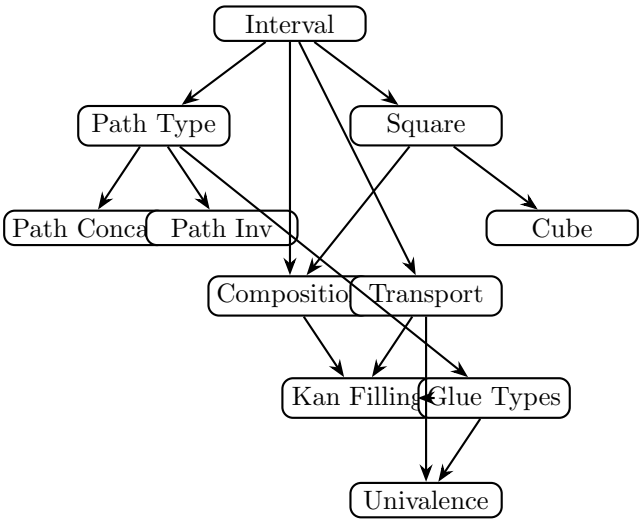
6 Dependency Graph Navigation

6.1 Motivation

The current sidebar is a flat list of categories. As the number of rule pages grows, students need a way to see *how concepts build on each other*. Which rules depend on the interval? Which rules use Glue types? The *Dependency Graph* provides a bird's-eye view of the system's structure and serves as an alternative navigation surface.

6.2 Graph Structure

Nodes are all pages in `cubical-viz`. Directed edges go from *dependency* to *dependent*: an edge $A \to B$ means "rule B uses concept A". The graph for the current set of pages is:



6.3 Visual Design

Layout. The graph is laid out using a *force-directed layout* (via a minimal D3-force simulation or a hand-rolled Fruchterman–Reingold iteration). Alternatively, since the graph is a DAG, a Sugiyama layered layout cleanly separates the levels (Interval at top, Univalence at bottom).

Nodes. Each node is a rounded rectangle with:

- The rule icon (same as sidebar).
- The rule name.
- A small color dot matching the rule's primary color in the 3D scene.
- Hover: a tooltip with the rule's one-sentence description.

Completed pages are shown in full color; pages not yet implemented are shown in gray with a "coming soon" badge.

Edges. Directed edges with arrowheads. Color: `rgba(255,255,255,0.25)` (dim white). On hover over a node, all edges to/from that node brighten to full white and direct dependencies are highlighted in **cyan** while dependents are highlighted in **orange**.

Background. Dark background (`#0d1117`), subtle dot-grid pattern.

6.4 Interaction

- **Click node**: Navigate to that rule page (equivalent to clicking the sidebar entry).
- **Hover node**: Show dependency highlights and tooltip.
- **Pan and zoom**: Mouse drag to pan; scroll wheel to zoom (via D3 zoom or custom transform).
- **Keyboard**: `/` focuses a search box that filters nodes by name.
- **"Show all paths to X"**: Right-click a node to highlight all transitive dependency paths leading to it, showing the "learning path" to that concept.

6.5 Access

The dependency graph is accessible via:

1. A new sidebar entry at the very top: **Overview** with a graph icon.
2. A "View in graph" link at the top of every rule page.
3. URL: `#overview` or `#graph`.

6.6 Implementation Notes

- New component: `src/lib/DependencyGraph.svelte`.
- New entry `'overview'` in `types.ts` under a new category **Overview** at the top of the sidebar.
- SVG-based rendering (not Three.js) for the graph itself.
- Dependency data: a static `DEPS` map in `types.ts`:

```
const DEPS: Partial<Record<VisualizationType,
                           VisualizationType[]>> = {
  'path-type-rule':    ['interval'],
  'square':            ['interval'],
  'composition-rule':  ['interval', 'square'],
  ...
};
```

- Force layout: 200 iterations of Fruchterman–Reingold on mount, then frozen (no continuous simulation to avoid battery drain).

7 Priority and Sequencing

Table 1 gives a suggested implementation order based on difficulty and pedagogical value.

Order	Feature	Effort	Rationale
1	Square / Cube (Sec. 3)	Medium	Fills existing stubs; high visual impact
2	Dependency Graph (Sec. 6)	Medium	Improves navigation for all existing pages
3	Face Lattice $\mathbb{F}$ (Sec. 4)	Medium	Unlocks understanding of partial elements
4	Proof Relevance Panel (Sec. 2)	Medium–Hard	Adds depth to all existing rules
5	Univalence Walkthrough (Sec. 5)	Hard	Flagship capstone; requires all others

Table 1: Suggested implementation order.

8 Conclusion

These five features complete the pedagogical arc of `cubical-viz`: from individual rules (Phase 1–2) to a fully connected, navigable, proof-aware learning environment. The most immediately impactful additions are the Square/Cube visualizations (filling existing stubs) and the Dependency Graph (making the system navigable as it grows). The Univalence Walkthrough serves as the capstone that ties everything together.